

Rapport d'Architecture – Étape 3 (Dylann André Fontus)

Répertoire GitHub Étape 3:
https://github.com/DylannFontus/Lab0_Log430/tree/main

Évolution vers Event Sourcing et CQRS

1. Introduction & objectifs

Ce document présente l'évolution majeure de l'architecture du système de gestion de point de vente (POS) à l'étape 3, marquée par l'adoption des patterns Event Sourcing et CQRS (Command Query Responsibility Segregation). Cette transformation prépare le système à une future migration vers une architecture microservices tout en améliorant significativement la traçabilité, la performance et la scalabilité.

L'objectif principal est d'implémenter un Event Store avec Redis et le pattern CQRS pour séparer les responsabilités de lecture et d'écriture, tout en maintenant la robustesse et la fiabilité du système existant.

2. Contraintes & exigences de qualité

Contraintes fonctionnelles

- **Respect du modèle métier existant** : gestion des magasins, stocks, ventes, produits, transferts logistiques
- **Interface web ergonomique** et responsive maintenue
- **Gestion multi-rôles** (caissier, logisticien, administrateur, maison mère)
- **Journalisation complète** des opérations via Event Store
- **Tests automatisés** étendus pour couvrir Event Sourcing et CQRS
- **API REST documentée** avec Swagger
- **Compatibilité ascendante** avec l'architecture existante

Nouvelles exigences Event Sourcing/CQRS

- **Immutabilité des événements** : tous les événements métier doivent être stockés de manière permanente
- **Reconstruction d'état** : possibilité de replay complet à partir de l'historique
- **Séparation Command/Query** : optimisation indépendante des opérations de lecture et d'écriture
- **Cohérence événementielle** : propagation fiable des changements d'état
- **Audit trail complet** : traçabilité de toutes les modifications

Exigences de qualité renforcées

- **Performance** :
 - Temps de réponse < 500ms pour les queries (optimisées via read models)
 - Temps de réponse < 1s pour les commands

- Débit d'événements > 1000 événements/seconde
 - **Sécurité** : gestion des accès, protection des données, conformité RGPD, audit complet
 - **Évolutivité** : architecture préparée pour les microservices, scalabilité horizontale
 - **Testabilité** : tests Event Sourcing, scénarios de replay, cohérence CQRS
 - **Observabilité** : métriques étendues pour Event Store et CQRS via Prometheus/Grafana
 - **Résilience** : tolérance aux pannes, recovery automatique via Event Store
-

3. Contexte & portée

Contexte métier inchangé

- **Acteurs** : caissier, client, logisticien, administrateur, maison mère
- **Systèmes tiers** : aucun actuellement (prévu : fournisseurs, ERP, systèmes externes)
- **Interfaces** : interface web (navigateur), API Django REST, documentation Swagger

Contexte technique évolué

- **Plateformes** : Linux, Docker, Docker Compose
- **Technologies core** : Python 3.11, Django 5.x, MySQL (état traditionnel)
- **Nouvelles technologies** :
 - **Redis Streams** : Event Store pour la persistance des événements
 - **Redis Pub/Sub** : Event Broker pour CQRS
 - **Event Sourcing** : pattern de persistance basé sur les événements
 - **CQRS** : séparation Command/Query avec read models optimisés
- **Technologies existantes maintenues** : NGINX, Redis (cache), Prometheus, Grafana, k6, DRF, Swagger

Architecture hybride

Le système adopte une **architecture hybride** combinant :

- **Persistance traditionnelle** (MySQL) pour la compatibilité
 - **Event Store** (Redis Streams) comme source de vérité événementielle
 - **Read models** optimisés pour les queries
 - **Event Broker** (Redis Pub/Sub) pour la communication asynchrone
-

4. Solution conceptuelle

Évolution architecturale majeure

Le système conserve ses trois sous-domaines métier mais adopte une nouvelle approche architecturale :

Sous-domaines métier (inchangés)

- **Ventes en magasin** : gestion du panier, finalisation/annulation des ventes, consultation du stock local
- **Gestion logistique** : transferts de stock, gestion du centre logistique, suivi des ruptures/surstocks
- **Supervision maison mère** : rapports, tableaux de bord, modification des produits

Nouvelle architecture Event Sourcing/CQRS

Flux Command (écriture) :

1. L'utilisateur déclenche une action métier (vente, transfert, modification)
2. Un **Command Handler** valide et traite la commande
3. Les événements métier sont persistés dans l'**Event Store** (Redis Streams)
4. Les événements sont publiés via l'**Event Broker** (Redis Pub/Sub)
5. Les **Event Handlers** mettent à jour les read models asynchrone

Flux Query (lecture) :

1. L'utilisateur demande des informations (consultation stock, rapports)
2. Les **Query Handlers** accèdent aux **read models optimisés**
3. Les données sont servies rapidement sans accès à l'Event Store
4. Les projections sont maintenues à jour via les événements

Événements métier principaux

- VenteCréée, VenteAnnulée, VenteFinalisée
 - StockTransféré, StockMisÀJour, StockRéapprovisionné
 - ProduitCréé, ProduitModifié, ProduitSupprimé
 - MagasinCréé, MagasinModifié
-

5. Vue statique (building block view)

Structure modulaire évoluée

Couche de présentation (inchangée)

- **Templates** : `templates/`, interface web maintenue
- **Vues web** : `views.py`, `urls.py`
- **API REST** : `api_views.py`, `serializers.py`, documentation Swagger

Nouvelle couche Command (CQRS)

- **Command Handlers** : `commands/` (ex: `CreateVenteCommandHandler`, `TransferStockCommandHandler`)
- **Command Dispatchers** : routage et validation des commandes
- **Aggregate Roots** : entités métier gérant leurs propres événements

Nouvelle couche Event Store

- **Event Store Service** : `services/EventStoreService.py`
- **Event Publisher** : `services/EventPublisher.py`
- **Event Models** : définition des événements métier typés

Nouvelle couche Query (CQRS)

- **Query Handlers** : `queries/` (ex: `StockQueryHandler`, `VenteQueryHandler`)
- **Read Models** : `read_models/` (projections optimisées)

- **Event Handlers** : `event_handlers/` (mise à jour des projections)

Couche logique existante (adaptée)

- **Services métiers** : `services/` (adaptés pour Event Sourcing)
- **Services hybrides** : gestion de la transition entre ancien et nouveau modèle

Couche de persistance hybride

- **MySQL** : persistance traditionnelle (phase de transition)
- **Redis Streams** : Event Store (source de vérité événementielle)
- **Redis Cache** : cache et read models
- **Modèles Django** : `models.py` (maintenus pour compatibilité)

Infrastructure Event Sourcing/CQRS

- **Event Broker** : Redis Pub/Sub pour la communication asynchrone
- **Projections Engine** : reconstruction automatique des read models
- **Replay Service** : reconstruction d'état via historique d'événements

Nouveaux composants techniques

- **Event Serializers** : sérialisation JSON des événements avec schémas versionnés
- **Consumer Groups** : distribution de charge via Redis Consumer Groups
- **Event Validators** : validation de la cohérence événementielle
- **Snapshot Service** : optimisation des agrégats via snapshots périodiques

6. Vue dynamique (runtime view)

Scénario clé : Vente en magasin avec Event Sourcing

Flux Command (création de vente)

1. **Interface utilisateur** : l'utilisateur finalise une vente via l'interface web
2. **Command Handler** : `CreateVenteCommandHandler` reçoit la commande
3. **Validation métier** : vérification du stock disponible, données client
4. **Génération d'événements** : création des événements `VenteCréée`, `StockMisÀJour`
5. **Persistence Event Store** : sauvegarde dans Redis Streams avec ID séquentiel
6. **Publication** : envoi des événements via Redis Pub/Sub
7. **Réponse synchrone** : confirmation immédiate à l'utilisateur

Flux asynchrone (mise à jour des projections)

1. **Event Handlers** : consommation des événements via Redis Consumer Groups
2. **Mise à jour read models** :
 - Projection stock actualisée
 - Statistiques de vente mises à jour
 - Cache invalidé/mis à jour

3. **Notification** : autres services notifiés du changement d'état

Flux Query (consultation optimisée)

1. **Query Handler** : `StockQueryHandler` traite une demande de consultation
2. **Accès read model** : lecture directe des projections optimisées
3. **Cache hit** : données servies depuis Redis cache si disponibles
4. **Réponse rapide** : < 100ms grâce aux read models pré-calculés

Scénario de replay (reconstruction d'état)

1. **Demande de replay** : reconstruction d'un agrégat ou d'une projection
2. **Event Store Query** : lecture séquentielle des événements depuis Redis Streams
3. **Projection rebuilding** : application séquentielle des événements
4. **State reconstruction** : reconstitution de l'état actuel
5. **Validation** : vérification de la cohérence avec l'état existant

Autres scénarios Event Sourcing

- **Transfert de stock** : événements `StockTransféré` entre magasins
- **Annulation de vente** : événement compensatoire `VenteAnnulée`
- **Audit trail** : consultation de l'historique complet d'un agrégat
- **Monitoring** : métriques Event Store via Prometheus

7. Vue de déploiement

Infrastructure étendue

Composants existants maintenus

- **Poste utilisateur** : navigateur web ou client API
- **NGINX** : reverse proxy HTTP (load balancing prévu)
- **Serveur applicatif** : conteneur Docker Django étendu avec Event Sourcing
- **MySQL** : base de données traditionnelle (phase de transition)
- **Prometheus/Grafana** : monitoring et visualisation

Nouveaux composants Event Sourcing

- **Redis Event Store** :
 - Instance Redis dédiée pour Event Store (Redis Streams)
 - Configuration AOF pour durabilité maximale
 - Clustering Redis prévu pour haute disponibilité
- **Redis Event Broker** :
 - Instance Redis pour Pub/Sub (peut être partagée)
 - Consumer Groups pour distribution de charge
- **Read Models Storage** :
 - Redis cache pour projections rapides
 - Possibilité d'ajouter des stores spécialisés (Elasticsearch, etc.)

Configuration Docker Compose étendue

```
services:
  web: # Django avec Event Sourcing
  db: # MySQL (transition)
  redis-cache: # Cache existant
  redis-eventstore: # Nouveau: Event Store
  redis-broker: # Nouveau: Event Broker
  prometheus: # Monitoring étendu
  grafana: # Dashboards Event Store
```

Monitoring étendu

- **Métriques Event Store** : nombre d'événements, latence, throughput
 - **Métriques CQRS** : performance commands vs queries, cohérence projections
 - **Alertes** : détection de lag dans les projections, anomalies événementielles
-

8. Vue des interfaces externes

Interfaces utilisateur (maintenues)

- **Interface web** : HTML/CSS/JS servie par Django via NGINX
- **API REST** : endpoints Django REST Framework, documentation Swagger

Nouvelles interfaces Event Sourcing

- **Event Store API** :
 - Endpoints pour consultation d'événements `/api/events/`
 - API de replay `/api/replay/{aggregate_id}`
 - Métriques Event Store `/metrics/eventstore`
- **CQRS API** :
 - Commands endpoints `/api/commands/{command_type}`
 - Queries endpoints `/api/queries/{query_type}`
 - Projections status `/api/projections/status`

Interfaces techniques

- **Redis Event Store** : accès via redis-py pour persistance événementielle
- **Redis Event Broker** : pub/sub via redis-py pour communication asynchrone
- **Event Serialization** : JSON avec schémas versionnés
- **Monitoring endpoints** : métriques Prometheus étendues

Interfaces de migration

- **Dual-write** : écriture simultanée ancien/nouveau modèle pendant la transition
- **Data migration** : outils de migration des données existantes vers Event Store
- **Compatibility layer** : maintien de la compatibilité avec l'API existante

9. Concepts transverses (crosscutting concepts)

Event Sourcing transversal

- **Immutabilité** : tous les événements sont immuables et horodatés
- **Idempotence** : les event handlers sont idempotents
- **Versioning** : gestion des versions d'événements pour évolutions futures
- **Concurrency** : gestion optimiste avec version checking

CQRS transversal

- **Séparation stricte** : commands ne retournent jamais de données, queries ne modifient jamais
- **Cohérence éventuelle** : accept du délai entre command et query
- **Projections** : read models optimisés pour chaque cas d'usage
- **Event-driven updates** : mise à jour asynchrone des projections

Sécurité renforcée

- **Audit trail complet** : traçabilité via Event Store
- **Event encryption** : chiffrement des événements sensibles
- **Access control** : permissions granulaires sur commands/queries
- **Data retention** : politiques de rétention pour conformité RGPD

Gestion des erreurs évoluée

- **Event validation** : validation stricte des événements avant persistance
- **Compensation** : événements compensatoires pour rollback
- **Dead letter queue** : gestion des événements en erreur
- **Circuit breaker** : protection contre les cascades d'erreurs

Tests Event Sourcing/CQRS

- **Event testing** : tests unitaires des event handlers
- **Projection testing** : validation de la cohérence des read models
- **Replay testing** : tests de reconstruction d'état
- **Integration testing** : tests end-to-end avec Event Store

10. Décisions architecturales

ADR 0004 : Event Store avec Redis

- **Choix** : Redis Streams pour Event Store
- **Justification** : performance, intégration existante, fonctionnalités natives
- **Impact** : foundation pour Event Sourcing, traçabilité complète

ADR 0005 : CQRS avec Event Broker Redis

- **Choix** : Redis Pub/Sub comme Event Broker pour CQRS

- **Justification** : cohérence avec Event Store, simplicité opérationnelle
- **Impact** : séparation Command/Query, performance optimisée

Justification de l'architecture hybride

- **Transition progressive** : migration sans interruption de service
- **Risk mitigation** : maintien de la persistance traditionnelle en parallèle
- **Learning curve** : adoption progressive des concepts Event Sourcing/CQRS
- **Performance validation** : comparaison des performances ancien/nouveau modèle

Choix technologiques Event Sourcing

- **Redis Streams** : append-only, ordering, persistence, consumer groups
- **JSON serialization** : lisibilité, debuggabilité, versioning
- **Python asyncio** : performance pour event handling asynchrone
- **Structured logging** : traçabilité et debugging améliorés

Patterns appliqués

- **Event Sourcing** : source de vérité événementielle
- **CQRS** : séparation Command/Query responsibility
- **Event-driven architecture** : communication asynchrone via événements
- **Saga pattern** : préparation pour workflows distribués futurs
- **Outbox pattern** : consistency entre Event Store et projections

Préparation microservices

- **Bounded contexts** : identification des limites métier via événements
- **Event contracts** : définition des interfaces entre services futurs
- **Distributed tracing** : préparation pour observabilité distribuée
- **Service mesh ready** : architecture compatible avec istio/envoy futur

11. Qualité, risques & dette technique

Métriques de qualité

Couverture de tests

- **Tests unitaires** : > 85% de couverture pour les event handlers et command handlers
- **Tests d'intégration** : couverture complète des flux Event Sourcing/CQRS
- **Tests de replay** : validation de la reconstruction d'état pour tous les agrégats
- **Tests de performance** : benchmarks Event Store (> 1000 événements/sec)
- **Tests de cohérence** : validation de la synchronisation read models

Métriques de performance

- **Latence commands** : < 1s (95e percentile)
- **Latence queries** : < 100ms (95e percentile)
- **Throughput Event Store** : > 1000 événements/seconde

- **Lag projections** : < 500ms entre événement et mise à jour read model
- **Disponibilité** : > 99.9% (objectif SLA)

Analyse des risques

Risques techniques

1. Single Point of Failure Redis

- **Impact** : Perte complète de l'Event Store
- **Probabilité** : Faible
- **Mitigation** : Clustering Redis, backup automatique, réplication maître-esclave

2. Cohérence éventuelle CQRS

- **Impact** : Données temporairement incohérentes entre commands et queries
- **Probabilité** : Moyenne (normale dans CQRS)
- **Mitigation** : Monitoring du lag, timeouts appropriés, UX adaptée

3. Croissance infinie Event Store

- **Impact** : Saturation stockage, dégradation performance
- **Probabilité** : Élevée à long terme
- **Mitigation** : Stratégie de rétention, archivage, snapshots

4. Complexité architecture hybride

- **Impact** : Bugs, maintenance difficile, courbe d'apprentissage
- **Probabilité** : Moyenne
- **Mitigation** : Documentation, formation équipe, migration progressive

Risques métier

1. Migration des données existantes

- **Impact** : Perte ou corruption de données historiques
- **Probabilité** : Faible
- **Mitigation** : Tests de migration, validation données, rollback plan

2. Performance dégradée pendant transition

- **Impact** : Expérience utilisateur dégradée
- **Probabilité** : Moyenne
- **Mitigation** : Tests de charge, monitoring continu, optimisations

3. Résistance au changement équipe

- **Impact** : Adoption lente, erreurs d'implémentation
- **Probabilité** : Moyenne
- **Mitigation** : Formation, documentation, support technique

Dettes techniques

Dettes techniques héritées

- **Modèles Django legacy** : coexistence avec Event Store nécessaire
- **Vues non-optimisées** : certaines vues accèdent encore directement à MySQL
- **Tests insuffisants** : couverture partielle des scénarios Event Sourcing
- **Monitoring limité** : métriques Event Store en cours d'implémentation

Stratégies d'amélioration continue

Code quality

- **Linting automatisé** : flake8, black, mypy pour cohérence code
- **Security scanning** : bandit, safety pour vulnérabilités
- **Dependency updates** : renovate pour mise à jour dépendances
- **Performance profiling** : py-spy, django-debug-toolbar

Architecture quality

- **Architecture Decision Records** : documentation de toutes les décisions
- **Code reviews** : validation architecture dans chaque PR
- **Refactoring planifié** : roadmap de remboursement dette technique
- **Prototype validation** : POC avant implémentation features majeures

12. Glossaire

Concepts Event Sourcing

Aggregate (Agréгат) : Entité métier qui encapsule la logique de domaine et génère des événements lors des changements d'état. Exemples : Vente, Stock, Produit.

Command (Commande) : Instruction qui exprime une intention de modifier l'état du système. Toujours nommée à l'impératif (ex: **CreateVente**, **TransferStock**).

Event (Événement) : Fait métier immutable qui s'est produit dans le passé. Toujours nommé au passé (ex: **VenteCréée**, **StockTransféré**).

Event Store : Base de données spécialisée dans le stockage séquentiel et immuable des événements métier. Dans notre cas : Redis Streams.

Event Stream : Séquence ordonnée d'événements liés à un agrégat spécifique, identifiée par un ID unique.

Replay : Processus de reconstruction de l'état d'un agrégat en appliquant séquentiellement tous ses événements historiques.

Snapshot : Image figée de l'état d'un agrégat à un moment donné, utilisée pour optimiser les performances de replay.

Concepts CQRS

Command Handler : Composant responsable du traitement d'une commande spécifique, de la validation métier et de la génération d'événements.

Command Side : Partie de l'architecture responsable des opérations d'écriture (commands) et de la modification d'état.

CQRS (Command Query Responsibility Segregation) : Pattern architectural qui sépare les responsabilités de lecture (Query) et d'écriture (Command) en utilisant des modèles différents.

Event Handler : Composant qui réagit à un événement spécifique pour mettre à jour les projections ou déclencher des actions.

Projection : Vue matérialisée des données optimisée pour un cas d'usage de lecture spécifique, construite à partir des événements.

Query Handler : Composant responsable du traitement d'une requête de lecture en accédant aux projections optimisées.

Query Side : Partie de l'architecture responsable des opérations de lecture (queries) via des modèles optimisés.

Read Model : Modèle de données dénormalisé et optimisé pour les opérations de lecture, distinct du modèle d'écriture.

Concepts techniques

AOF (Append Only File) : Mode de persistance Redis qui journalise toutes les opérations d'écriture pour garantir la durabilité.

Consumer Group : Mécanisme Redis Streams permettant la distribution de charge entre plusieurs consommateurs d'événements.

Event Broker : Système de messagerie responsable de la propagation des événements entre les composants. Dans notre cas : Redis Pub/Sub.

Event-driven Architecture : Style architectural où les composants communiquent principalement via des événements asynchrones.

Idempotence : Propriété d'une opération qui peut être exécutée plusieurs fois sans changer le résultat au-delà de la première exécution.

Redis Pub/Sub : Mécanisme de messagerie Redis basé sur le pattern Publisher/Subscriber pour la communication temps réel.

Redis Streams : Structure de données Redis conçue pour le stockage et le traitement de flux de données en temps réel.

Concepts métier POS

Bounded Context : Limite explicite à l'intérieur de laquelle un modèle de domaine particulier est défini et applicable.

Centre Logistique : Entrepôt central gérant le stock global et les transferts vers les magasins individuels.

Magasin : Point de vente physique avec son propre stock local et ses opérations de caisse.

Panier : Collection temporaire de produits sélectionnés par un client avant finalisation de la vente.

Point de Vente (POS) : Système de gestion des ventes au détail intégrant caisse, gestion de stock et supervision.

Stock : Quantité de produits disponibles dans un magasin ou centre logistique à un moment donné.

Transfert de Stock : Opération de déplacement de produits entre le centre logistique et un magasin, ou entre magasins.

Vente : Transaction commerciale entre un client et un magasin, matérialisée par un ticket et une modification de stock.

Acronymes et abréviations

ADR : Architecture Decision Record - Document de décision architecturale **API** : Application Programming Interface **CORS** : Cross-Origin Resource Sharing **DRF** : Django REST Framework **JSON** : JavaScript Object Notation **ORM** : Object-Relational Mapping **POC** : Proof of Concept **REST** : Representational State Transfer **SLA** : Service Level Agreement **SQL** : Structured Query Language **UML** : Unified Modeling Language **UUID** : Universally Unique Identifier